Build a Serverless
Lead Capture on AWS

A full-stack serverless architecture using S3, Lambda, API Gateway, DynamoDB, SES, CloudFront & Route 53

Static Hosting | Serverless | CDN | Email Notifications | Data Storage

Ahmed Tetteh

Project Overview

Project Goal

Build a serverless lead capture system for an eBook website. When a visitor clicks "Download Ebook", their contact details are captured, stored in DynamoDB, and an email notification is sent to the admin via SES — all without managing any servers.

Tech Stack

HTML, CSS, JavaScript

Scale

500 active users, 10–15 downloads/day

Global Performance

CloudFront CDN delivery

Data Storage

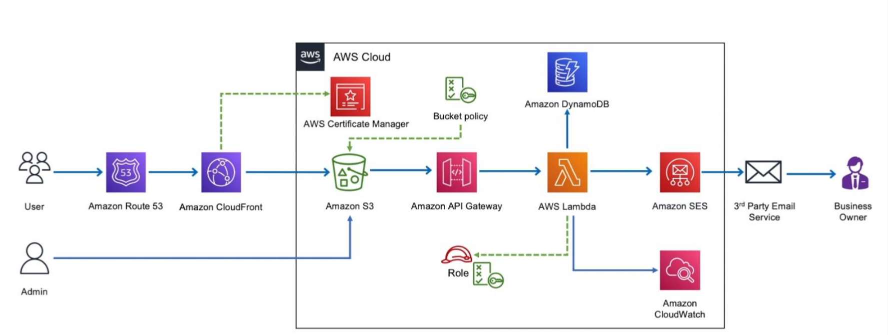
Contact details in DynamoDB

Email Notifications

On every download via SES

Security

HTTPS enforced on all requests



Project Architecture

AWS Services Used

Phase	Service	Description
Phase 1	**Amazon S3**	Static website hosting + bucket policies
Phase 1	**CloudFront**	Global CDN for low-latency delivery
Phase 1	**Route 53**	DNS management & custom domain
Phase 1	**ACM**	SSL/TLS certificate issuance
Phase 2	**API Gateway**	REST API endpoint trigger
Phase 2	**AWS Lambda**	Serverless compute + email dispatch
Phase 2	**Amazon SES**	Email notifications to admin
Phase 3	**DynamoDB**	NoSQL storage for contact details
All	**IAM**	Roles, policies & permissions
All	**CloudWatch**	Logging & monitoring

Phase 1 Phase 2 Phase 3 All Phases

Architecture Flow

User Browser
HTML/CSS/JS
S3 + CloudFront

Route 53 + CloudFront
Custom domain
HTTPS via ACM

API Gateway
REST endpoint
CORS enabled

AWS Lambda
Node.js handler
Business logic

Request Flow Summary

- 1. User visits eBook site served from S3 via CloudFront
- 2. Route 53 routes domain → CloudFront distribution (HTTPS)
- 3. User fills form and clicks Download; JS triggers AJAX call
- 4. AJAX calls API Gateway endpoint (POST /contact)
- 5. API Gateway invokes Lambda function with form payload
- 6. Lambda sends email via SES + stores data in DynamoDB
- 7. CloudWatch logs all Lambda executions

Amazon SES
Email to admin

DynamoDB
Store contact details

CloudWatch
Logs & monitoring

IAM Roles
Permissions & policies

PHASE 1

Static Website Hosting

01 Create S3 Bucket

Created an S3 bucket and configured it for static website hosting. Initially faced access restrictions — resolved by writing a bucket policy to allow public read access to objects.

02 Upload Website Files

Used the AWS CLI command `aws s3 sync . s3://bucketName` to upload all HTML, CSS, and JS files. Created a custom 404 error page for missing routes.

03 Request SSL Certificate (ACM)

Requested a public certificate via AWS Certificate Manager for the custom domain. Validated it by creating a CNAME record in Route 53. Note: CloudFront only accepts certificates from us-east-1.

04 Configure CloudFront Distribution

Created a CloudFront distribution pointing to the S3 bucket. Attached the ACM certificate to enforce HTTPS. CloudFront caches content globally for low-latency delivery.

05 Route 53 DNS Setup

Created a Hosted Zone in Route 53 and added an A record (Alias) pointing the custom domain to the CloudFront distribution. Updated nameservers on the domain registrar.

Phase 1 — Challenges & Key Learnings

⚠ Challenge

Route 53 A record alias not pointing to S3

→

✓ Resolution

The S3 bucket name must exactly match the domain name for direct S3 hosting with Route 53 Alias records.

⚠ Challenge

Route 53 couldn't see the S3 bucket endpoint

→

✓ Resolution

Switched to using CloudFront as the origin instead of S3 directly — this also unlocks global caching.

⚠ Challenge

CloudFront couldn't find the ACM certificate

→

✓ Resolution

ACM certificates for CloudFront must be issued in us-east-1 (N. Virginia), regardless of where your resources are deployed.

⚠ Challenge

Mapping nameservers from AWS Hosted Zone

→

✓ Resolution

Copied the 4 NS records from Route 53 Hosted Zone and replaced the nameservers at the domain registrar.

Best Practice: Always use CloudFront in front of S3 — it decouples domain naming, adds HTTPS, and provides global caching.

PHASE 2

Serverless Backend — Contact Form

01 SES — Email Identity Verification

Created and verified two email identities in Amazon SES: one sender (noreply@domain.com) and one admin receiver. Required for sandbox mode.

02 Lambda Function (Node.js)

Wrote a Lambda function that receives form data, constructs an email payload, and calls SES.sendEmail(). Used test-driven events to validate the function before connecting API Gateway.

03 API Gateway — REST Endpoint

Created a REST API with a POST /contact route. Enabled CORS so the front-end JavaScript (hosted on S3/CloudFront) can make cross-origin requests.

04 Connecting the Pieces

The website's JavaScript uses an AJAX call on form submit → hits API Gateway → triggers Lambda → Lambda calls SES. Verified end-to-end with real form submission.

Lambda Handler (Node.js) - Sample Code

```
const AWS = require('aws-sdk');
const ses = new AWS.SES();

exports.handler = async (event) => {
  const body = JSON.parse(event.body);
  const { name, email } = body;

  await ses.sendEmail({
    Destination: {
      ToAddresses: [ADMIN_EMAIL]
    },
    Message: { Subject: {...},
      Body: { Text: { Data:
        `New lead: ${name} ${email}`
      }}},
    Source: SENDER_EMAIL
  }).promise();
  return { statusCode: 200 };
};
```

PHASE 3

Data Management — DynamoDB Integration

✓ Create DynamoDB Table

Created a DynamoDB table to store lead capture data. Table includes fields for name, email, timestamp, and source.

✓ IAM Inline Policy for Lambda

Added an inline policy to the Lambda execution role granting dynamodb:PutItem permission on the target table. Inline policy is preferred here — it's deleted automatically with the role.

✓ Update Lambda Function

Extended the Lambda handler to call DynamoDB.putItem() with the contact details in addition to sending the SES email.

✓ End-to-End Verification

Submitted test forms and confirmed: (1) admin receives email via SES, (2) DynamoDB table is populated, (3) CloudWatch logs show both operations succeeded.

DynamoDB Schema

email	String (PK)	Partition key
name	String	
timestamp	String	ISO 8601
source	String	eBook / campaign

IAM Inline Policy (Lambda Role)

```
{ "Effect": "Allow",
  "Action": "dynamodb:PutItem",
  "Resource": "arn:aws:dynamodb:
    region:account:table/leads"
}
```

Results & Monitoring

3

Phases Completed

Hosting → Backend → Data

9+

AWS Services Used

S3, CF, R53, ACM, GW, λ, SES, DB, CW

0

Servers Managed

Fully serverless architecture

HTTPS

All Requests Secured

ACM + CloudFront enforcement

Verified End-to-End Results

- Static eBook website live at custom domain with HTTPS (CloudFront + Route 53 + ACM)
- Contact form submits successfully — AJAX → API Gateway → Lambda — with 200 response
- Admin receives email notification via SES for every download request
- Contact details (name, email, timestamp) stored in DynamoDB after each submission
- CloudWatch log streams confirm Lambda executions from both test events and API Gateway calls
- S3 and CloudFront correctly handle GET requests; POST requests are handled by API Gateway only

Project Summary

This project demonstrates a production-grade serverless architecture on AWS. Starting from a plain HTML/CSS/JS eBook website, the solution progressively adds HTTPS delivery via CloudFront, serverless contact capture via Lambda + API Gateway, email notifications via SES, and persistent storage via DynamoDB — all without provisioning or managing any servers.

Infrastructure

→ S3 Bucket Policies

→ CloudFront CDN

→ Route 53 DNS

→ ACM TLS Certs

Serverless

→ Lambda (Node.js)

→ API Gateway REST

→ CORS Configuration

→ SES Email

Data & Security

→ DynamoDB NoSQL

→ IAM Roles & Policies

→ CloudWatch Logs

→ Inline Policies